LBiC Computing Resources: Quick Start Guide

Accessing the lab

- **Building**: Access to the G2 block (LBiC's location) requires student ID ("RA"). If your ID fails, please contact prof. Von Zuben* for authorization (CTIC).
- **LBiC**: The lab access control is done through a regular key. If you plan to use the physical space, kindly ask Prof. Von Zuben for a copy of the lab key.

Accessing the computers

If you don't have your credentials yet, send an email† requesting it. To access hosts remotely, connect via VPN then SSH into the host. If you are locally connected to the lab's network, only an SSH connection needed.

- **VPN**: Connect to the lab VPN using your assigned credentials. A comprehensive setup guide was included in your credential email for reference.
- **SSH:** After establishing the VPN connection, log in to the PC and/or server:

```
ssh username@ip_address
```

Hosts

- **Workstations**: There are a few dedicated PCs available for local or remote research. Please contact us† to request a machine for your project's duration.
- **Servers**: Experiments are currently hosted on a shared primary server. Please schedule your sessions in advance via the resource reservation spreadsheet.

hostname	paubrasil.lbic.fee.unicamp.br
IP	143.106.45.179
CPU	AMD Ryzen 9 9950X (16 cores)
GPU	NVIDIA RTX 5070 Ti (16 GiB)
RAM	128 GiB DDR5 @ 4400 MT/s

On workstations, `root` privileges are granted (and recommended), and users are responsible for system maintenance. On servers, `root` access is restricted; please contact us† for any required system-wide package installations.

* vonzuben[at]unicamp[dot]br

† vonzuben[at]unicamp[dot]br, with copy to ito[dot]rafael[at]gmail[dot]com

General commands

Change Password

Please change your password after your first login.

```
passwd
```

Monitoring CPU and GPU processes

```
htop                # for memory usage and CPU cores utilization
nvtop               # for GPU proccesses and VRAM memory utilization
```

Help

```
man <command>       # example: "man podman"
<command> --help    # example: "mamba --help"
```

Terminal multiplexer (tmux)

Prevent script termination on SSH disconnect by running `tmux` in the background.

Start a new session

```
tmux new -s my_experiment
```

Detach (leave it running)

Press ctrl+b ⟶ d

Reattach later

```
tmux attach -t my_experiment
```

Split screen

Vertical: ctrl+b ⟶ %
Horizontal: ctrl+b ⟶ "
Switch panes: ctrl+b ⟶ ← ↑ ↓ →

List active sessions

```
tmux ls
```

Kill a session

```
tmux kill-session -t my_experiment
```

Zoom pane (toggle fullscreen)

Press ctrl+b ⟶ z

Scroll terminal history

Press ctrl+b ⟶ [

Use arrows/PgUp to scroll. Press q to exit.

v1.2 | March 6, 2026

Mamba (Python environments)

What is it? Mamba is a package manager. It is a drop-in replacement for Conda, but rewritten in C++ to be drastically faster. It isolates your Python versions and libraries so different projects don't conflict.

Create a new environment

Specify the Python version and any starting packages.

```
mamba create -n <environment_name> python=3.10 pytorch -c pytorch
```

List all environments

See all available environments on the server.

```
mamba env list
```

Activate the environment

Activate the environment every time you log in before running your code.

```
mamba activate <environment_name>
```

Install additional packages

Package installation can be done with `mamba` or `pip` (after environment activation).

```
mamba install pandas  # mixing mamba and pip can create dependencies issues
pip install scipy     # if you need to use both, prioritize for mamba over pip
```

Deactivate the environment

Return to the default system environment.

```
mamba deactivate
```

Delete an environment

Clean up when your experiment is finished.

```
mamba env remove -n <environment_name>
```

Example: train an AI model on MNIST dataset

```
mamba create -n mnist_env python=3.10 -y
mamba activate mnist_env
pip install torch torchvision --upgrade --index-url https://download.pytorch.org/whl/cu128
wget -P ~/mnist_test https://raw.githubusercontent.com/pytorch/examples/main/mnist/main.py
cd mnist_test
python main.py --epochs 1
```

Podman (containers)

What is it? Podman is a container engine. It is functionally identical to Docker (you can even use the alias "docker" to run the commands), but it is "daemonless" and runs securely without needing root privileges.

Run a container interactively

Download an image, mount volumes, and run a command inside the container.

```
podman run --rm -it --device nvidia.com/gpu=all \
       --volume /dataset/mnist:/dataset         \
       --volume $(pwd):/workspace               \
       pytorch/pytorch:latest                   \
       bash
```

List running containers and downloaded images

```
podman ps             # check what is currently running and get the container IDs
podman image ls       # check what container images are currently stored locally
```

Enter inside a running container

```
podman exec -it <container_id> bash
```

Stop a container

```
podman stop <container_id>
```

Remove a stopped container

```
podman rm <container_id>
```

Example: run a Jupyter Notebook (with GPU)

Run a Jupyter container, map port `<xxxx>`, and keep it running in the background. If it asks for a token, run `podman logs <container_id>` to find the login URL.

```
podman run -d --device nvidia.com/gpu=all -p <xxxx>:8888 jupyter/scipy-notebook
```

Run this on your local machine to map the server's Jupyter port to your own computer.

```
ssh -L <xxxx>:localhost:<xxxx> -Nf username@server_ip
```

Then access `https://localhost:<xxxx>` on a web browser.

v1.2 | March 6, 2026